

基于SigLIP的小规模图文检索系统 — 中期报告

1. 任务概述

本项目基于 SigLIP (Sigmoid Loss for Language-Image Pretraining) 模型, 在 Flickr8k 数据集上实现了一个小规模图文检索系统。与 CLIP 使用 softmax 进行全局归一化不同, SigLIP 对每个图文对独立使用 sigmoid 判断是否匹配 (二分类), 对 batch size 更鲁棒。

2. 代码实现

2.1 TODO 1: SigLIP 损失函数 (loss.py)

实现了 `SigLIPLoss.forward()`, 计算流程如下:

- L2 归一化:** 对图像嵌入和文本嵌入分别做 `F.normalize(x, dim=-1)`, 使向量长度为 1, 余弦相似度范围为 $[-1, 1]$ 。
- 余弦相似度矩阵:** `logits = image_embeds @ text_embeds.T`, 得到 $[B, B]$ 的矩阵, 第 (i, j) 个元素表示第 i 张图与第 j 条文本的余弦相似度 z_{ij} 。
- 缩放 + 偏置:** `logits = -t * logits + b`, 其中 $t = \text{logit_scale.exp}()$ 为可学习温度系数 (初始值 10.0), b 为可学习偏置 (初始值 -10.0)。温度系数用于放大相似度差异, 偏置用于调整匹配的基准线。存储 $\log(t)$ 而非直接存储 t , 是因为 t 必须为正数, 而 $\exp()$ 的输出恒为正, 无需额外约束。
- 匹配标签:** 对角线 $s_{ij} = +1$ (正样本对), 其余 $s_{ij} = -1$ (负样本对)。使用 `labels = 2 * torch.eye(B) - 1` 构造。
- 逐对 sigmoid loss:** `loss = F.softplus(labels * logits)`, 即 $\log(1 + \exp(s_{ij} \cdot (-t \cdot z_{ij} + b)))$ 。使用 `F.softplus` 替代 `torch.log1p(torch.exp(...))` 数值更稳定。
- 取均值:** `loss.sum() / B`, 即除以 $|B|$, 符合 README 中的公式:

$$\mathcal{L}_{\text{SigLIP}} = \frac{1}{|B|} \sum_{i=1}^{|B|} \sum_{j=1}^{|B|} \log(1 + \exp(s_{ij}(-t z_{ij} + b)))$$

2.2 TODO 2: ResNet BasicBlock (models/resnet_custom.py)

在 `BasicBlock.__init__()` 中填写了卷积层和 BN 层:

- `conv1`: 3×3 卷积, `in_channels` $\rightarrow$ `out_channels`, stride 由参数传入 (可能为 2 用于下采样), `bias=False` (因为紧跟 BN)
- `bn1`: BatchNorm2d, `out_channels` 个通道
- `conv2`: 3×3 卷积, `out_channels` $\rightarrow$ `out_channels`, stride 固定为 1, `bias=False`

- `bn2` : BatchNorm2d, `out_channels` 个通道
- `relu` : ReLU(inplace=True)
- `downsample` : 由外部 `_make_layer` 传入, 当主路改变了通道数或空间尺寸时, 捷径需要通过 `downsample` (1×1 卷积+BN) 对齐形状, 否则残差连接 `out + identity` 无法相加

forward 的残差连接逻辑为: `out = relu(bn2(conv2(relu(bn1(conv1(x)))))) + identity`, 注意 `relu` 放在加完 `identity` 之后。

2.3 TODO 3: Transformer 文本编码器 forward (models/transformer_encoder.py)

实现了 `TransformerTextEncoder.forward()`, 流程为:

1. **Token embedding**: `self.token_embedding(input_ids)` 将 [B, 32] 的 ID 序列映射为 [B, 32, 256] 的嵌入矩阵。
2. **加位置编码**: `self.positional_encoding(x)` 在嵌入上叠加 sin/cos 位置指纹, 让模型能区分同一词在不同位置的语义差异。
3. **创建 padding mask**: `input_ids.eq(self.padding_idx)` 找出哪些位置是 PAD (True 表示忽略), 传给 Transformer 的 `src_key_padding_mask` 参数。
4. **Transformer 编码**: `self.encoder(x, src_key_padding_mask=padding_mask)`, 2 层 Pre-LN TransformerEncoder, 4 头注意力, FFN 隐藏维度 1024。
5. **均值池化**: 将 [B, 32, 256] 压缩为 [B, 256]。关键是用 `mask = (~padding_mask).unsqueeze(-1).float()` 标记有效位置, 对有效 token 的嵌入求和后除以有效 token 数量, 忽略 PAD 位置。
6. **线性投影**: `self.proj(x)` 映射到共享语义空间。

2.4 TODO 4: 检索评估函数 (train_siglip.py)

实现了 `evaluate_retrieval()`, 计算 Text-to-Image 和 Image-to-Text 的 Recall@1/5/10:

1. **收集所有嵌入**: 遍历整个验证/测试集, 收集所有图像嵌入、文本嵌入、`image_id` 和 `caption`。
2. **L2 归一化**: 对图像和文本嵌入做归一化。
3. **按 image_id 分组**: Flickr8k 中每张图像有 1-5 条 caption, 用 `defaultdict` 将同一 `image_id` 对应的文本索引分组。
4. **聚合图像嵌入**: 对同一图像的多个 embedding 取均值, 得到每个 unique 图像的代表, 再重新归一化。这是因为同一图像的不同 caption 会产生不同的图像 embedding (由于 batch 内不同的上下文), 取均值可以得到更稳定的图像表示。
5. **计算相似度矩阵**: `sim = unique_image_embeds @ text_embeds.T`, 形状为 [n_unique_img, N_text]。

6. **Text-to-Image 检索**：对每条文本，按相似度排序所有 unique 图像，检查正确图像是否在 Top-K 中。
7. **Image-to-Text 检索**：对每张图像，按相似度排序所有文本，检查 Top-K 中是否包含任意一条正确 caption（因为同一图像有 5 条 caption，只要命中一条就算成功）。

3. 训练配置

两个模型共享相同的图像编码器（ResNet18, width=32）和嵌入维度（256），仅文本编码器不同：

参数	MLP 模型	Transformer 模型
数据集	Flickr8k	Flickr8k
文本编码器	MLP (embedding → mean pooling → 2层MLP)	Transformer (2层, 4头, Pre-LN)
图像编码器	ResNet18 (width=32)	ResNet18 (width=32)
嵌入维度	256	256
训练轮数	100	200
Batch size	64	64
学习率	3e-4	3e-4
优化器	AdamW (weight_decay=1e-4)	AdamW (weight_decay=1e-4)
梯度裁剪	max_norm=1.0	max_norm=1.0
GPU	NVIDIA RTX 4090	NVIDIA RTX 4090

训练命令：

```
# MLP 文本编码器
python train_siglip.py \
  --data-dir Flickr8k \
  --text-encoder mlp \
  --epochs 100 \
  --batch-size 64 \
  --output-dir outputs/resnet_mlp

# Transformer 文本编码器

python train_siglip.py \
  --data-dir Flickr8k \
  --text-encoder transformer \
  --epochs 200 \
  --batch-size 64 \
  --output-dir outputs/resnet_transformer
```

4. 训练结果

4.1 最终测试集指标对比

指标	MLP	Transformer
t2i_R@1	26.67%	26.74%
t2i_R@5	50.96%	47.55%
t2i_R@10	61.07%	57.22%
i2t_R@1	27.99%	28.88%
i2t_R@5	52.40%	49.76%
i2t_R@10	63.01%	59.51%
test_loss	2.9449	3.9094

4.2 MLP vs Transformer 对比分析

出乎意料的是，MLP 文本编码器的检索指标与 Transformer 基本持平，在 R@5 和 R@10 上甚至更优。这一现象看似反直觉，但在当前实验设定下是合理的，原因如下：

1. 数据集规模太小，Transformer 容易过拟合

Flickr8k 训练集仅约 6000 对图文，词表大小约 5000。Transformer 的参数数量远大于 MLP（自注意力权重 + FFN + 投影层），在小数据集上更容易记忆训练样本而非学习泛化特征。这从 test_loss 的差异可以看出：MLP 的 test_loss 为 2.94，Transformer 为 3.91，说明 Transformer 的泛化能力确实更弱。

2. 短文本场景下均值池化已经够用

Flickr8k 的 caption 平均仅约 10 个词，且多为简单描述（如 "a dog is running on grass"）。在这种短文本场景下，均值池化已经能捕捉到关键词信息（"dog"、"grass"），Transformer 的上下文建模优势无法充分体现。MLP 将 "a dog running on grass" 和 "a cat running on grass" 区分开主要依赖关键词 "dog"/"cat" 的嵌入差异，而非上下文关系，而这在短文本中已经足够。

3. 对比学习主要依赖关键词匹配

在当前的小规模对比学习框架下，模型最有效的策略是学会将图像中的视觉概念与文本中的关键词对齐（如 "dog" 对应狗的视觉特征），而非理解复杂的句法结构。MLP 的均值池化天然适合这种关键词匹配模式。

4. R@1 vs R@5/R@10 的差异

值得注意的是，Transformer 在 R@1 上略优于 MLP (26.74% vs 26.67% 的 t2i, 28.88% vs 27.99% 的 i2t)，说明 Transformer 在精确检索（只看第 1 名）时能利用少量上下文信息做出更准确的判断。但 MLP 在 R@5 和 R@10 上明显更好 (t2i_R@5: 50.96% vs 47.55%, t2i_R@10: 61.07% vs 57.22%)，说明 MLP 学到的嵌入空间整体排序质量更高，而 Transformer 由于过拟合导致部分样本的排序出现偏差。

5. 分析与优化方向

5.1 当前结果总体评价

- 两个模型在 Flickr8k 测试集上的 t2i_R@1 均约 27%，远高于随机猜测的 0.1%（1000 张图中随机选 1 张），说明模型确实学到了有意义的图文对齐能力。
- R@10 达到 57%-63%，意味着在 Top-10 检索结果中有较大概率找到正确匹配，具备一定的实用价值。
- 两个模型的 i2t_R@1 均略高于 t2i_R@1，这是因为 Image-to-Text 检索中同一图像有 5 条正确 caption，命中概率更高。

5.2 当前主要瓶颈：过拟合

从训练过程可以观察到 Transformer 模型存在明显的过拟合现象（train_loss 远低于 val_loss），这是当前性能的主要瓶颈。过拟合的原因：

- **数据量不足**：Flickr8k 训练集仅约 6000 对图文，模型容量相对于数据量过大
- **缺乏正则化**：当前训练没有使用学习率调度，学习率始终为 3e-4，后期参数更新幅度仍然很大

5.3 后续优化方向

1. **学习率调度**：添加 Cosine Annealing Scheduler，让学习率从 3e-4 按余弦曲线逐渐衰减到接近 0，使后期参数更新更温和，减少过拟合。
2. **增大 batch size**：对比学习依赖大量负对来提供有效的训练信号。当前 batch_size=64 只产生 64×63=4032 个负对，增大到 128 或 256 可以让模型在每个 batch 中见到更多不匹配的图文组合，学到更鲁棒的表示。
3. **早停策略**：当验证集的 R@1 连续多个 epoch 不再提升时停止训练，避免过度拟合训练集。当前代码已保存 best_siglip.pt（基于 val_r1 最优），可直接使用 best checkpoint 的结果。
4. **增大模型容量 + 更强正则化**：尝试增大 image-width 和 embed-dim，同时配合更大的 weight_decay 和 dropout 来抵消过拟合，让模型在不记忆训练数据的前提下学到更丰富的特征。

5. **Top-K 检索可视化**: 编写文本→图像检索示例的可视化代码，展示检索成功和失败的案例，直观理解模型的偏好和局限。
6. **预训练模型可视化**: 运行 visualization/app.py，体验 Google 预训练 SigLIP 模型的图文交叉相似度矩阵，与自训练模型进行对比。

附录：代码实现

A.1 SigLIP 损失函数 (loss.py — SigLIPLoss.forward)

```
def forward(self, image_embeds: torch.Tensor, text_embeds: torch.Tensor)
-> torch.Tensor:
    # 1. L2 归一化
    image_embeds = F.normalize(image_embeds, dim=-1)
    text_embeds = F.normalize(text_embeds, dim=-1)

    # 2. 余弦相似度矩阵 [B, B]
    logits = image_embeds @ text_embeds.T

    # 3. 缩放 + 偏置
    t = self.logit_scale.exp() # 可学习温度
    b = self.logit_bias        # 可学习偏置
    logits = -t * logits + b

    # 4. 匹配标签: 对角线 +1, 其余 -1
    B = logits.size(0)
    labels = 2 * torch.eye(B, device=logits.device) - 1

    # 5. 逐对 sigmoid loss
    loss = labels * logits
    loss = F.softplus(loss) # log(1 + exp(s * (-t*z + b)))

    # 6. 除以 |B|
    return loss.sum() / B
```

A.2 ResNet BasicBlock (models/resnet_custom.py — BasicBlock.init)

```
def __init__(self, in_channels, out_channels, stride=1,
downsample=None):
    super().__init__()
    self.conv1 = nn.Conv2d(in_channels, out_channels,
                           kernel_size=3, stride=stride, padding=1,
                           bias=False)
    self.bn1 = nn.BatchNorm2d(out_channels)
    self.conv2 = nn.Conv2d(out_channels, out_channels,
```

```

        kernel_size=3, stride=1, padding=1, bias=False)
self.bn2 = nn.BatchNorm2d(out_channels)
self.relu = nn.ReLU(inplace=True)
self.downsample = downsample

```

A.3 Transformer 文本编码器 (models/transformer_encoder.py — TransformerTextEncoder.forward)

```

def forward(self, input_ids):
    # 1. Token embedding
    x = self.token_embedding(input_ids) # [B, L, D]

    # 2. 加位置编码
    x = self.positional_encoding(x)

    # 3. 创建 padding mask (True = 忽略)
    padding_mask = input_ids.eq(self.padding_idx) # [B, L]

    # 4. Transformer 编码
    x = self.encoder(x, src_key_padding_mask=padding_mask) # [B, L, D]

    # 5. 均值池化 (忽略 padding 位置)
    mask = (~padding_mask).unsqueeze(-1).float() # [B, L, 1]
    x = (x * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1) # [B, D]

    # 6. 线性投影
    x = self.proj(x) # [B, D]
    return x

```

A.4 检索评估函数 (train_siglip.py — evaluate_retrieval)

```

@torch.no_grad()
def evaluate_retrieval(model, loader, device, topk=(1, 5, 10)):
    model.eval()
    all_image_embeds, all_text_embeds = [], []
    all_image_ids, all_captions = [], []

    for batch in loader:
        images = batch["image"].to(device)
        input_ids = batch["input_ids"].to(device)
        image_embeds, text_embeds = model(images, input_ids)
        all_image_embeds.append(image_embeds)
        all_text_embeds.append(text_embeds)
        all_image_ids.extend(batch["image_id"])
        all_captions.extend(batch["caption"])

```

```

image_embeds = torch.cat(all_image_embeds) # [N, D]
text_embeds = torch.cat(all_text_embeds) # [N, D]

# 归一化
image_embeds = F.normalize(image_embeds, dim=-1)
text_embeds = F.normalize(text_embeds, dim=-1)

# 按 image_id 分组 (同一图像有多个 caption)
from collections import defaultdict
img2indices = defaultdict(list)
img2text = defaultdict(list)
for idx, img_id in enumerate(all_image_ids):
    img2indices[img_id].append(idx)
    img2text[img_id].append(idx)
unique_images = list(img2indices.keys())
img_idx_map = {img_id: i for i, img_id in enumerate(unique_images)}

# 聚合同一图像的多个 embedding (取均值)
unique_image_embeds = []
for img_id in unique_images:
    indices = img2indices[img_id]
    unique_image_embeds.append(image_embeds[indices].mean(dim=0))
unique_image_embeds = torch.stack(unique_image_embeds) #
[n_unique_img, D]
unique_image_embeds = F.normalize(unique_image_embeds, dim=-1)

# 相似度矩阵
sim = unique_image_embeds @ text_embeds.T # [n_unique_img, N_text]

# Text-to-Image 检索: 对每个文本, 找最相似的图像
t2i_hits = {k: 0 for k in topk}
for text_i in range(len(all_captions)):
    img_id = all_image_ids[text_i]
    target_img_idx = img_idx_map[img_id]
    scores = sim[:, text_i]
    ranked = scores.argsort(descending=True)
    for k in topk:
        if target_img_idx in ranked[:k].tolist():
            t2i_hits[k] += 1

# Image-to-Text 检索: 对每个图像, 找最相似的文本
i2t_hits = {k: 0 for k in topk}
for img_idx, img_id in enumerate(unique_images):
    gt_text_indices = img2text[img_id]
    scores = sim[img_idx]
    ranked = scores.argsort(descending=True)
    for k in topk:
        if any(t in ranked[:k].tolist() for t in gt_text_indices):
            i2t_hits[k] += 1

```



```
n_text = len(all_captions)
n_img = len(unique_images)
metrics = {}
for k in topk:
    metrics[f"t2i_R@{k}"] = t2i_hits[k] / n_text
    metrics[f"i2t_R@{k}"] = i2t_hits[k] / n_img
return metrics
```